Explications détaillées

Programme `pretty_printers.py`

1. Présentation générale

Ce script Python est une petite démonstration de **contrôle de l'affichage dans un terminal**. Il utilise le module `time` pour temporiser, et des **séquences d'échappement ANSI** pour déplacer le curseur et effacer du texte à l'écran.

Le programme enchaîne trois étapes :

- un compte à rebours de 5 à 1 affiché sur une seule ligne ;
- deux questions posées à l'utilisateur, dont l'affichage est effacé après chaque réponse ;
- un message récapitulatif visible 2 secondes, remplacé par un message d'au revoir.

Code complet

```
import time

for i in range(5, 0, -1):
   print(f"Compte à rebours : {i}", end="\r", flush=True)
   time.sleep(1)
print("Terminé !          ")

val = input("Choisissez une valeur entière entre 0 et 4: ")
print("\033[1A\033[J", end="")
val1 = input("Choisissez une valeur entière entre 5 et 10: ")
print("\033[1A\033[J", end="")   # remonte de 2 lignes et efface jusqu'en bas

print(f"Vous avez choisi d'abord {val}, puis {val1}", end="\r", flush=True)
time.sleep(2)
print("\033[K" + "A bientôt.")   # efface la ligne avant d'écrire
```

2. Le compte à rebours

```
for i in range(5, 0, -1):
   print(f"Compte à rebours : {i}", end="\r", flush=True)
   time.sleep(1)
print("Terminé !          ")
```

Élément	Rôle
`range(5, 0, -1)`	Parcourt les valeurs 5, 4, 3, 2, 1 (départ 5, arrêt avant 0, pas de -1).
`end="\r"`	Au lieu de passer à la ligne, revient au **début de la même ligne** (retour chariot). Le chiffre suivant s'écrit par-dessus le précédent : un seul compteur qui change.
`flush=True`	Force l'affichage immédiat. Sans saut de ligne, le texte resterait sinon dans la mémoire tampon.
`time.sleep(1)`	Attend 1 seconde entre deux chiffres.
`print("Terminé ! ")`	Les espaces finaux **écrasent** les caractères restants de « Compte à rebours : 1 », qui est plus long que « Terminé ! ».

Idée clé : \r ramène le curseur au début de la ligne, mais n'efface rien. Si le nouveau texte est plus court que l'ancien, il reste des caractères de l'ancien : d'où les espaces ajoutés à la fin.

3. Les deux saisies avec effacement

```
val = input("Choisissez une valeur entière entre 0 et 4: ")
print("\033[1A\033[J", end="")
val1 = input("Choisissez une valeur entière entre 5 et 10: ")
print("\033[1A\033[J", end="")
```

`input()` affiche la question, attend la réponse, puis l'utilisateur appuie sur Entrée : le curseur descend d'une ligne. Le script **remonte alors d'une ligne** et **efface** tout ce qui se trouve à partir de là. La question et la réponse disparaissent de l'écran.

Séquence	Effet
`\033[1A`	Remonte le curseur de **1 ligne**.
`\033[J`	Efface de la position du curseur **jusqu'à la fin de l'écran**.

Remarques sur le code

- Le commentaire `# remonte de 2 lignes` est inexact : `1A` ne remonte que d'**une** ligne.
- Les valeurs ne sont **pas vérifiées** : `val` et `val1` sont des chaînes de caractères, rien n'impose qu'elles soient des entiers ni qu'elles respectent les intervalles annoncés (0-4 et 5-10).

4. Le message récapitulatif et l'au revoir

```
print(f"Vous avez choisi d'abord {val}, puis {val1}", end="\r", flush=True)
time.sleep(2)
print("\033[K" + "A bientôt.")
```

- Le message s'affiche, puis le curseur revient au début de la ligne (`\r`) sans la quitter.
- Il reste visible pendant 2 secondes (`time.sleep(2)`).
- `\033[K` **efface la ligne courante** (de la position du curseur jusqu'à la fin de la ligne), puis « A bientôt. » s'écrit à la place. Sans cet effacement, des morceaux de la phrase précédente, plus longue, resteraient visibles.